

Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет компьютерных систем и сетей

Кафедра электронных вычислительных машин

Дисциплина: Операционные системы и системное программирование

ОТЧЕТ
по лабораторной работе №5
на тему
“Потоки исполнения, взаимодействие и синхронизация”

Выполнил:
студ. гр. 450502 Бурш А.П.

Проверил:
пр. Благиных В.Д.

Минск, 2026

1 ЦЕЛЬ ЛАБОРАТОРНОЙ РАБОТЫ

Две лабораторных в одной:

5.1) Аналогична лабораторной 4, но только с потоками, posix-семафорами и мьютексом в рамках одного процесса. Дополнительно обрабатывается ещё две клавиши – увеличение и уменьшение размера очереди. Следует предусмотреть обработку запроса на уменьшение очереди таким образом, чтобы при появлении пустого места уменьшался размер очереди, а не очередной производитель размещал там своё сообщение.

5.2) Аналогична лабораторной № 5.1, но с использованием условных переменных (см. лекции СПОВМ/ОСисП).

2 ВЫПОЛНЕНИЕ РАБОТЫ

2.1 Описание алгоритма выполнения работы

Программа состоит из основного процесса, который в зависимости от выбора создаёт очередь сообщений с семафорами или условными переменными, после чего ожидает и обрабатывает нажатия клавиш, порождая и завершая процессы двух типов — производители и потребители. Производители формируют сообщения и, если в очереди есть место, перемещают их туда. Потребители, если в очереди есть сообщения, извлекают их оттуда, обрабатывают и освобождают память с ними связанную. Также есть возможно уменьшить или увеличить размер очереди сообщений и вывести её состояние. Программа обрабатывает ошибки и корректно освобождает ресурсы.

2.2 Описание основных функций

Программа состоит из нескольких частей, представляющих собой некоторые функции. К ним относятся функции:

- 1) Основная функция лабораторной работы;
- 2) Вывод статуса очереди сообщений с семафорами;
- 3) Вывод статуса очереди сообщений с условными переменными;
- 4) Производитель с семафорами;
- 5) Производитель с условными переменными;
- 6) Генерация сообщений;
- 7) Вычисление контрольных данных сообщения;
- 8) Потребитель с семафорами;
- 9) Потребитель с условными переменными;
- 10) Перепроверка контрольных данных;
- 11) Инициализация очереди сообщений с семафорами;
- 12) Удаление очереди сообщений с семафорами;
- 13) Добавление в очередь сообщений с семафорами;
- 14) Извлечение из очереди сообщений с семафорами;
- 15) Инициализация очереди сообщений с условными переменными;

- 16) Удаление очереди сообщений с условными переменными;
- 17) Добавление в очередь сообщений с условными переменными;
- 18) Извлечение из очереди сообщений с условными переменными.

2.2.1 Функция runLab()

Данная функция как параметры принимает функции для работы с очередью сообщений. И в зависимости от выбора в стартовом меню запускает работу с очередью сообщений с семафорами или условными переменными.

2.2.2 Функция showStatusSem()

Функция выводит состояние очереди сообщений с семафорами.

2.2.3 Функция showStatusCond()

Данная функция выводит состояние очереди сообщений с условными переменными.

2.2.4 Функция producerSem()

Данная функция представляет собой процесс производителя, который генерирует сообщения и добавляет их в очередь сообщений с семафорами.

2.2.5 Функция producerCond()

Функция представляет собой процесс производителя, который генерирует сообщения и добавляет их в очередь сообщений с условными переменными.

2.2.6 Функция produceMessage()

Функция генерирует сообщение, заполняя его структуру случайными значениями.

2.2.7 Функция calculateHash()

Функция считает контрольные данные сообщения по его первым size + 1 байтам.

2.2.8 Функция consumerSem()

Данная функция представляет собой процесс потребителя, который генерирует сообщения и добавляет их в очередь с семафорами.

2.2.9 Функция consumerCond()

Функция представляет собой процесс потребителя, который генерирует сообщения и добавляет их в очередь с условными переменными.

2.2.10 Функция consumeMessage()

Функция перерасчитывает контрольные данные поглощаемого сообщения, соотнося их с имеющимися.

2.2.11 Функция initQueueSem()

Функция инициализирует очередь сообщений с семафорами.

2.2.12 Функция deleteQueueSem()

Функция удаляет очередь сообщений с семафорами.

2.2.13 Функция addToQueueSem()

Функция добавления сообщения в очередь сообщений с семафорами.

2.2.14 Функция getFromQueueSem()

Функция извлечения из очереди сообщений с семафорами.

2.2.15 Функция initQueueCond()

Функция инициализации очереди сообщений с условными переменными.

2.2.16 Функция deleteQueueCond()

Функция удаления очереди сообщений с условными переменными.

2.2.17 Функция addToQueueCond()

Данная функция добавляет сообщение в очередь сообщений с условными переменными.

2.2.18 Функция getFromQueueCond()

Функция извлечения из очереди сообщений с условными переменными.

3 ОПИСАНИЕ ПОРЯДКА СБОРКИ

Проект собирается с помощью системы CMake, используя файл CMakeLists.txt, который описывает правила сборки. Для сборки создаётся отдельная папка build, если она ещё не существует, после чего в ней генерируются необходимые файлы и выполняется компиляция исходного кода с формированием исполняемого файла. Такой подход позволяет отделить исходные файлы от результатов сборки, поддерживать порядок в проекте и упрощает управление его структурой.

Для очистки проекта можно использовать команду `cmake --build build --target clean`, которая удаляет скомпилированные файлы из каталога build. Это помогает поддерживать чистоту рабочей среды и обеспечивает корректность последующих сборок.

Если хочешь, я могу сразу переписать весь фрагмент в более формальном стиле для отчёта.

4 РЕЗУЛЬТАТЫ РАБОТЫ ПРОГРАММЫ

```

=====+
| LAB 05 - THREAD QUEUE CONTROL |
| Producers, consumers, semaphores, condition variables |
=====+
| Select a mode: |
| [1] Lab 5.1 POSIX semaphores |
| [2] Lab 5.2 Condition variables |
| [q] Exit |
=====+
[Select mode] > 1
=====+
| Lab 5.1 - POSIX Semaphores |
=====+
| Queue initialized with capacity 10 |
=====+
| Commands: |
| [p] add producer [P] remove producer [+] grow queue |
| [c] add consumer [C] remove consumer [-] shrink queue |
| [s] show status [q] return to main menu |
=====+
| Status snapshot: POSIX semaphores |
=====+
| Capacity: 10 |
| Occupied: 0 |
| Free slots: 10 |
| Load: [...] 0/10 (0%) |
| Total added: 0 |
| Total extracted: 0 |
| Producers: 0 |
| Consumers: 0 |
=====+
| Semaphore filledSlots: 0 |
| Semaphore emptySlots: 10 |
=====+
[Command] > +
[Increase by] > 15
[Command] > p
[Command] > p
[Command] > p
[Command] > c
[Command] > s
=====+
| Status snapshot: POSIX semaphores |
=====+
| Capacity: 25 |
| Occupied: 10 |
| Free slots: 15 |
| Load: [#####.....] 10/25 (40%) |
| Total added: 15 |
| Total extracted: 5 |
| Producers: 3 |
| Consumers: 1 |
=====+
| Semaphore filledSlots: 10 |
| Semaphore emptySlots: 15 |
=====+
[Command] > q

```

Рисунок 1 – Увеличение размера очереди и запуск нескольких производителей и потребителей для очереди сообщений с семафорами

```

=====+
| LAB 05 - THREAD QUEUE CONTROL |
| Producers, consumers, semaphores, condition variables |
=====+
| Select a mode: |

```

```

| [1] Lab 5.1 POSIX semaphores |
| [2] Lab 5.2 Condition variables |
| [q] Exit |
+-----+
[Select mode] > 2
+-----+
| Lab 5.2 - Condition Variables |
+-----+
| Queue initialized with capacity 10 |
+-----+
| Commands: |
| [p] add producer [P] remove producer [+] grow queue |
| [c] add consumer [C] remove consumer [-] shrink queue |
| [s] show status [q] return to main menu |
+-----+
+-----+
| Status snapshot: Condition variables |
+-----+
| Capacity: 10 |
| Occupied: 0 |
| Free slots: 10 |
| Load: [...] 0/10 (0%) |
| Total added: 0 |
| Total extracted: 0 |
| Producers: 0 |
| Consumers: 0 |
+-----+
[Command] > -
[Decrease by] > 7
[Command] > p
[Command] > p
[Command] > p
[Command] > c
[Command] > c
[Command] > s
+-----+
| Status snapshot: Condition variables |
+-----+
| Capacity: 3 |
| Occupied: 3 |
| Free slots: 0 |
| Load: [#####] 3/3 (100%) |
| Total added: 7 |
| Total extracted: 4 |
| Producers: 3 |
| Consumers: 2 |
+-----+
[Command] > q
Failed to destroy mutex: Success

```

Рисунок 2 – Уменьшение размера очереди и запуск нескольких производителей и потребителей для очереди сообщений с условными переменными

ЗАКЛЮЧЕНИЕ

При выполнении лабораторной работы были разработаны процессы-производители, которые генерируют сообщение и если есть место в очереди, помещают их туда, и процессы-потребители, которые извлекают сообщения из очереди, если они там есть, а также основной процесс, в котором происходит выбор, с какой очередью работать: с семафорами или условными переменными. А затем начинается создание и удаление потребителей и производителей, выводится информация о состоянии очереди сообщений, изменяется размер очереди сообщений.